COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	10/21/2026	**DATE SUBMITTED:**	10/21/2026
NAME:	Barizo, Julian Benedict Macatangay, Justin Jake	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 14 **[Algorithm Complexities]**

1. PROCEDURE OUTPUT

Procedure (Theoretical)

	Base Case	Worst Case	Analysis
Algorithm 1 (Insertion)	$O(n)$	$O(n^2)$	**Best case occurs when the array is already sorted, requiring n-1 comparisons and 0 shifts. While, worst case occurs when the array is in reverse order, requiring n^2 / 2 comparisons and element shifts.**
Algorithm 2 (Bubble)	$O(n)$	$O(n^2)$	**In the best case with an already sorted array, Bubble Sort makes a single linear pass of n-1 comparisons with 0 swaps. In the worst case where the array happens to be reversed, it makes** $\frac{n(n-1)}{2}$ **comparisons and swaps, yielding quadratic time complexity O(n^2)**

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

Procedure (Experimental)

sortingAlgo.h

```cpp
#ifndef SORTINGALGO_H
#define SORTINGALGO_H

#include <cstddef>
#include <utility>

template<typename T>
void bubbleSort(T arr[], size_t arrSize) {
    for (size_t i = 0; i < arrSize - 1; i++) {
        bool swapped = false;
        for (size_t j = 0; j < arrSize - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                std::swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        // If no elements were swapped in inner loop, array is sorted
        if (!swapped) break;
    }
}

template <typename T>
void insertionSort(T arr[], size_t arrSize) {
    for (size_t k = 1; k < arrSize; k++) {
        T temp = arr[k];
        int j = k - 1;

        while (j >= 0 && arr[j] > temp) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = temp;
    }
}

#endif
```

main.cpp

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <thread>
#include <chrono>
#include <string>
#include <mutex>
#include "sortingAlgo.h"

std::mutex coutMutex;

template <typename Func, typename T>
void runAndMeasure(std::string name, Func sortFunc, T arr[], size_t size) {
    auto start = std::chrono::high_resolution_clock::now();

    sortFunc(arr, size);

    auto end = std::chrono::high_resolution_clock::now();
    auto elapsed = std::chrono::duration_cast<std::chrono::nanoseconds>(end - start);

    std::lock_guard<std::mutex> lock(coutMutex);
    std::cout << name << " took: " << elapsed.count() << " ns\n";
}

void testCases(size_t n) {
    std::cout << "\n== N = " << n << "==\n";

    int* arr1 = new int[n];
    int* arr2 = new int[n];

    // --- BEST CASE: Already Sorted (0 to n-1) ---
    for (size_t i = 0; i < n; i++) {
        arr1[i] = i;
        arr2[i] = i;
    }
```

```
    std::cout << "--- BEST CASE ---\n";
    std::thread t1(runAndMeasure<void(int[], size_t), int>, "Bubble Sort", bubbleSort<int>, arr1, n);
    std::thread t2(runAndMeasure<void(int[], size_t), int>, "Insertion Sort", insertionSort<int>, arr2, n);
    t1.join();
    t2.join();

    // --- WORST CASE: Reverse Sorted (n-1 down to 0) ---
    for (size_t i = 0; i < n; i++) {
        int val = n - i - 1;
        arr1[i] = val;
        arr2[i] = val;
    }

    std::cout << "--- WORST CASE ---\n";
    std::thread t3(runAndMeasure<void(int[], size_t), int>, "Bubble Sort", bubbleSort<int>, arr1, n);
    std::thread t4(runAndMeasure<void(int[], size_t), int>, "Insertion Sort", insertionSort<int>, arr2, n);
    t3.join();
    t4.join();

    delete[] arr1;
    delete[] arr2;
}

int main() {
    // Run tests for all three required sizes
    testCases(100);
    testCases(1000);
    testCases(10000);

    return 0;
}
```

	Algorithm 1 (Insertion)	Algorithm 2 (Bubble)

Best Case	**N = 100**
	`Insertion Sort took: 500 ns`
	N = 1000
	`Insertion Sort took: 3400 ns`
	N = 10000
	`Insertion Sort took: 42100 ns`
		N = 100
		`Bubble Sort took: 400 ns`
		N = 1000
		`Bubble Sort took: 2200 ns`
		N= 10000
		`Bubble Sort took: 27100 ns`

Worst Case	**N = 100**
	`Insertion Sort took: 12900 ns`
	N = 1000
	`Insertion Sort took: 1150200 ns`
	N = 10000
	`Insertion Sort took: 119101400 ns`
		N = 100
		`Bubble Sort took: 26700 ns`
		N = 1000
		`Bubble Sort took: 2731100 ns`
		N = 10000
		`Bubble Sort took: 265256800 ns`

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

‘

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Supplementary

ILO A: Contrast and compare the two searching techniques used in class. The required deliverable for this activity would be the theoretical and empirical analysis.

For the theoretical analysis, the deliverable must be:

- Pseudocode Analysis
- Time and Space Complexities

For the empirical analysis, the deliverable must be the filled-out table and your analysis.

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000		
10,000		
100,000		

Provide your analysis.

Pseudocode

Linear Search

```
ALGORITHM LinearSearch(array, size, target)
    FOR i FROM 0 TO size - 1 DO
        IF array[i] == target THEN
            RETURN i
        END IF
    END FOR
    RETURN -1
END ALGORITHM
```

Binary Search

```
ALGORITHM BinarySearch(array, size, target)
    low = 0
    high = size - 1

    WHILE low <= high DO
        mid = low + (high - low) / 2
        IF array[mid] == target THEN
            RETURN mid
        ELSE IF array[mid] > target THEN
            high = mid - 1
        ELSE
            low = mid + 1
        END IF
    END WHILE

    RETURN -1
END ALGORITHM
```

Code Screenshots

searchingAlgo.h

```
#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

#include <iostream>

template <typename T>
void linearSearchAR(int size, T dataFind, T array[]) {
    for (int j = 0; j < size; j++) {
        if (dataFind == array[j]) {
            // std::cout << "Searching is successful\n";
            return;
        }
    }
    // std::cout << "Searching is unsuccessful\n";
}

template <typename T>
void binarySearchArr(T data[], int arrSize, T dataFind) {
    int low = 0, high = arrSize - 1, middle;
    while (low <= high) {
        middle = low + (high - low) / 2;
        if (dataFind == data[middle]) {
            // std::cout << "the data was found\n";
            return;
        }
        else if (dataFind < data[middle]) {
            high = middle - 1;
        }
        else {
            low = middle + 1;
        }
    }
    // std::cout << "The data was not found\n";
}

#endif
```

supplementary.cpp

```cpp
#include <iostream>
#include <thread>
#include <chrono>
#include <mutex>
#include <string>
#include "searchingAlgo.h"

std::mutex coutMutex;

// Helper 1: For Binary Search signature (arr, size, target)
template <typename Func, typename T>
void runBinarySearch(std::string name, Func searchFunc, T arr[], int size, T target) {
    auto start = std::chrono::high_resolution_clock::now();

    searchFunc(arr, size, target);

    auto end = std::chrono::high_resolution_clock::now();
    auto elapsed = std::chrono::duration_cast<std::chrono::nanoseconds>(end - start);

    std::lock_guard<std::mutex> lock(coutMutex);
    std::cout << name << " took: " << elapsed.count() << " ns\n";
}

// Helper 2: For Linear Search signature (size, target, arr)
template <typename Func, typename T>
void runLinearSearch(std::string name, Func searchFunc, int size, T target, T arr[]) {
    auto start = std::chrono::high_resolution_clock::now();

    searchFunc(size, target, arr);

    auto end = std::chrono::high_resolution_clock::now();
    auto elapsed = std::chrono::duration_cast<std::chrono::nanoseconds>(end - start);

    std::lock_guard<std::mutex> lock(coutMutex);
    std::cout << name << " took: " << elapsed.count() << " ns\n";
}

void testSearchWorstCase(int n) {
    std::cout << "\n================== Input Size N = " << n << " ==================\n";

    int* arr = new int[n];

    for (int i = 0; i < n; i++) {
        arr[i] = i;
    }

    int target = -1; // Value not present in array forces worst case
```

```
    // Lines 37 & 38: Correct parameter orders and explicit template types (int)
    std::thread t1(runBinarySearch<void(int[], int, int), int>, "Binary Search", binarySearchArr<int>, arr, n, target);
    std::thread t2(runLinearSearch<void(int, int, int[]), int>, "Linear Search", linearSearchAR<int>, n, target, arr);

    t1.join();
    t2.join();

    delete[] arr;
}

int main() {
    testSearchWorstCase(1000);
    testSearchWorstCase(10000);
    testSearchWorstCase(100000);

    return 0;
}
```

Time and Space Complexities

Algorithm	Best time	Worst time	Space Complexity	Analysis
Linear Search	**O(1)**	**O(N)**	**O(1)**	Best case occurs when target is at index 0. Worst case occurs when target is at index N-1 or missing, requiring N comparisons. Operates iteratively in place using constant space.
Binary Search	**O(1)**	**O(log N)**	**O(1)**	Best case occurs when target is at the middle index. Worst case occurs when target is absent or at extremities, halving the search space repeatedly. Requires sorted input and constant space.

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	Binary Search took: 100 ns	Linear Search took: 2400 ns
10,000	Binary Search took: 200 ns	Linear Search took: 21700 ns
100,000	Binary Search took: 400 ns	Linear Search took: 223300 ns

Conclusion

Barizo

I've learnt about the implementation and empirical analysis of sorting and searching algorithms using multithreading and the chrono library. Working with thread functions showcased how parallel execution can speed up tasks while requiring careful console synchronization to avoid messy output. Benchmarking using the theoretical and experimental analysis demonstrated that theoretical complexities like linear scaling in linear search directly reflect real world execution times as input sizes grow. Overall, this activity emphasized how choosing the right algorithm and data structures drastically optimizes performance for large datasets.

Macatangay

Through this activity, I used Big-O notation and C++ benchmarks to verify how algorithmic complexity governs performance as input size grows. Although Insertion and Bubble Sort share $O(n)$ best-case and $O(n^2)$ worst-case bounds, Insertion Sort proved faster experimentally due to fewer swaps, while Binary Search ($O(\log n)$) vastly outperformed Linear Search ($O(n)$) on large inputs. Moving forward, I aim to refine my profiling techniques to minimize system timing variance during microsecond measurements.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Generation and Guidance of the <chrono> and <thread> library**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.